청약 플랫폼 데이터 구조

지금 DB에 어떤 데이터가 어떤 경로로 들어와 어떤 테이블에 쌓이는지를 현재 상태 기준으로 정리한 문서입니다. (변경 이력은 "청약플랫폼_DB_가이드.pdf" 참고)

기준일	2026-10-02 (LH 전체 기간 수집 완료 후)
브랜치	`jiyoon(cheongyakP)` 커밋 `3dc0adf` 까지 (main 미반영)
DB	PostgreSQL `cheongyak` (localhost:5432), 약 1.65GB — 테이블 13개 + 뷰 4개
원칙	**원본 DB**는 API 응답을 필터·계산 없이 그대로 저장하고, **가공 레이어(뷰)**가 원본을 읽어 서비스용 값을 계산합니다.

목차

1 **한눈에 보기**
3층 구조(원본 → 공고문에서 읽은 값 → 가공 레이어)와 테이블별 행 수

2 **데이터가 들어오는 흐름**
출처 5곳 → 수집 코드 → 테이블, 매일 오전 6시 실행 순서

3 **원본 테이블** 원본
listings · historical_results · lh_notice_details · 대출 · 통계 2종 · sh_notices — 1행의 의미, 키, 주요 컬럼

4 **공고문·첨부에서 읽은 테이블** 추출
notice_documents · lh_rental_houses · notice_costs · notice_option_costs — 계약금과 근거 원문

5 **가공 레이어** 가공
listing_features · recommendable_listings · cutoff_stats · competition_stats

6 **테이블 연결 방법**
어떤 키로 조인하는지, 자주 쓰는 SQL

7 **갱신 규칙과 알아둘 점**
수집 완료 상태, 지켜야 할 규칙, 구조상 한계

1. 한눈에 보기

출처
청약홈 분양정보·경쟁률
청약홈 통계 2종
LH 공고 API 3종
파인 대출 3종
SH 홈페이지 (API 없음)

→

① 원본 테이블 API 응답 그대로
`listings` 공고 × 주택형
`historical_results` 경쟁률·당첨가점·특별공급
`lh_notice_details` LH 상세·공급 원본 JSON
`loan_products` · `loan_rate_options` 대출
`bankbook_stats` · `subscription_stats` 통계
`sh_notices` SH 게시판 글

↓ 공고문 PDF · 주택목록 엑셀 (접수 예정·접수 중 공고만)

② 공고문·첨부에서 읽은 값 근거 원문 줄 포함
`notice_documents` 공고문 텍스트
`lh_rental_houses` 매입임대 주택별 임대조건
`notice_costs` 주택형별 계약금 + 근거 줄
`notice_option_costs` 옵션 계약금 원문

→

③ 가공 레이어 뷰
`listing_features`
평·평당가·평형 구간
`recommendable_listings`
접수 예정·접수 중 + 계약금
`cutoff_stats`
커트라인 통계
`competition_stats`
경쟁률 통계

↓
추천 · 보유자산 조회 API

126,082
listings — 공고 × 주택형 (청약홈 25,860 · LH 100,200 · SH 22)

346,328
historical_results — 경쟁률 68,402 · 당첨가점 29,766 · 특별공급 248,160

2,044,091
통계 — 청약통장 2,041,305 + 청약 신청·당첨자 2,786

21,327
lh_notice_details — LH 상세·공급정보 원본 (2010-10 ~ 현재, 전부 완료)

967
notice_costs — 주택형별 계약금 (근거 원문 줄 포함)

841
recommendable_listings — 지금 추천 대상 (공고 119건)

층	테이블	행	컬럼	크기	1행의 의미
원본	`listings`	126,082	57	404MB	공고 1건의 주택형 1개
	`historical_results`	346,328	23	719MB	공고 × 주택형 × (순위·지역 또는 특공 종류)의 결과 1건
	`lh_notice_details`	21,327	7	83MB	LH 공고 1건의 목록·상세·공급 API 응답 원본
	`loan_products`	280	27	0.6MB	대출 상품 1개
	`loan_rate_options`	682	13	0.6MB	상품의 금리 옵션 1개 (담보·상환·금리 유형)
	`bankbook_stats`	2,041,305	16	359MB	월 × 지역 × 통장 종류(× 평형 × 가입기간)의 계좌 수
	`subscription_stats`	2,786	18	1MB	월(× 시도)의 연령대별 신청·당첨 건수 또는 경쟁률
	`sh_notices`	2,621	13	4.6MB	SH 분양·임대 게시판 글 1건
	`user_profiles`	0	12	—	서비스 사용자 1명 (수집 대상 아님, 사용자가 입력)
추출	`notice_documents`	302	10	5.7MB	공고 1건의 공고문 텍스트 (또는 주택목록 읽기 상태)
	`lh_rental_houses`	1,692	19	1.4MB	LH 주택목록 엑셀의 주택 1호
	`notice_costs`	967	11	0.7MB	공고 × 주택형의 계약금 (listings 행과 1:1)
	`notice_option_costs`	219	10	0.2MB	분양 공고문의 옵션 가격표 1줄

가공	`listing_features` 구체화 뷰	126,082	—	63MB	listings 1행 + 계산 컬럼
	`recommendable_listings` 뷰	841	56	—	접수 예정·접수 중인 listing_features 행 + 계약금
	`cutoff_stats` 구체화 뷰	2,344	—	0.4MB	지역 × 시군구 × 평형 × 민영여부 × 거주구분 × 연도의 커트라인 통계
	`competition_stats` 구체화 뷰	9,676	—	1.4MB	유형 × 지역 × 시군구 × 평형 × 구분 × 연도의 경쟁률 통계

2. 데이터가 들어오는 흐름

2-1. 출처별 수집 내용

출처	무엇을 (API 기능)	범위	들어가는 테이블	수집 코드 (`app/data_pipeline/`)
청약홈 분양정보 ApplyhomeInfoDetailSvc	공고 상세 + 주택형 × 5개 유형 (APT, 오피스텔·도시형, 무순위·잔여, 공공지원민간임대, 임의공급) = 10개	전체 기간 (2020-02~) 공고 6,065건	`listings` (source=applyhome)	`applyhome_client.py`
청약홈 경쟁률 ApplyhomeInfoCmpetRtSvc	경쟁률 × 5개 유형 + 당첨가점 + 특별공급 신청현황 = 7개	전체 기간	`historical_results`	`competition_client.py`
청약홈 통계 ApplyhomeBnkbStatSvc ApplyhomeStatSvc	청약통장 가입현황 3개 (전체·통장별·가입기간별) 청약 신청·당첨자 5개 (지역별·연령별 신청자/당첨자, 지역별 경쟁률)	2010-01~ / 2015-01~ 2020-02~	`bankbook_stats` `subscription_stats`	`stats_client.py`
LH data.go.kr B552555	공고 목록 + 공고별 상세정보 + 공급정보 = 3개 (분양·임대·주거복지·신혼희망)	API가 주는 전체 (2010-10~) 공고 21,327건	`lh_notice_details` (원본 JSON) `listings` (source=lh)	`lh_client.py`
파인 금감원 금융상품비교공시	주택담보대출 · 전세자금대출 · 개인신용대출 × 금융권역 5개	공시 중인 상품 전체	`loan_products` `loan_rate_options`	`finlife_client.py`
SH 홈페이지 (API 없음)	분양·임대 게시판 목록 + 상세 페이지 (본문·첨부 링크)	게시판 전체 2,621건	`sh_notices` 모집공고 → `listings` (source=sh)	`sh_client.py`
공고문 · 첨부 PDF · 엑셀	접수 예정·접수 중 공고의 공고문 PDF, LH 주택목록 엑셀, LH 공급가격 PDF, SH 주택목록 PDF	추천 대상 공고 119건	`notice_documents` · `lh_rental_houses` · `notice_costs` · `notice_option_costs`	`document_client.py` `house_list_client.py`

모든 DB 저장은 `save_listings.py` 한 파일에서 합니다 (API 호출은 `*_client.py`, 저장은 `save_*` 함수). 테이블 구조는 `app/models/`, 뷰는 `app/db/views.py`.

2-2. 매일 오전 6시 실행 순서 (`collect_all`)

#	단계	하는 일	갱신 방식
1	`save_applyhome_listings`	청약홈 공고 × 주택형 전체	전체를 다시 받아 같은 키는 갱신, 새 행은 추가
2	`save_historical_results`	경쟁률 · 당첨가점 · 특별공급 전체	전체를 다시 받아 갱신·추가
3	`save_lh_listings`	LH 목록 전체 → 새 공고와 **목록 행이 바뀐 공고**만 상세·공급 API 호출	최신 공고부터. 100건 연속 실패(일일 한도)면 멈추고 다음 날 이어서
4	`save_sh_notices`	SH 게시판 새 글	새 글만 상세 페이지 읽음, 기존 글은 제목·조회수 갱신
5	`save_sh_recruitments`	최근 90일 SH 모집공고의 공고문에서 접수기간 → listings	공고문은 한 번만 내려받음
6	`save_loan_products`	파인 대출 상품 + 금리 옵션	상품 갱신, 옵션은 지우고 다시 넣음
7	`save_applyhome_stats`	청약통장 통계 + 청약 신청·당첨자 통계	저장된 마지막 달부터 다시 받음 (월별 자료)
8	`refresh_views`	구체화 뷰 3개 다시 계산	원본 기준으로 전부 재계산
9	`save_notice_costs`	추천 대상 공고의 공고문·주택목록 읽기 → 계약금·옵션 계약금	안 읽은 공고문만 내려받고, 계약금은 저장된 텍스트로 매번 다시 계산

> 스케줄러는 **backend 컨테이너가 켜져 있을 때** 매일 06:00(Asia/Seoul)에 실행됩니다 (`scheduler.py`). 한 단계가 실패해도 나머지는 계속 진행하고, 수동 실행은 `docker compose exec backend python -m app.data_pipeline.save_listings` 입니다. 2026-10-01, 10-02 오전 6시 자동 수집이 정상 실행된 것을 확인했습니다. LH API가 요청 과다(429)로 잠시 차단하면 10·30·60초 쉬었다가 다시 시도합니다.

3. 원본 테이블

> **원본 원칙** — API 응답을 행 필터·계산·추정 없이 저장합니다. 자주 쓰는 필드는 컬럼으로 옮기고, **응답 행 전체는 `raw` (JSON) 컬럼에 그대로** 남겨서 컬럼에 없는 필드나 나중에 생기는 필드도 보존됩니다. DBeaver에서 직접 고치지 마세요 (다음 수집 때 API 값으로 덮어씁니다).

3-1. listings — 공고 × 주택형 (126,082행, 57컬럼)

키: `(source, external_id, model_no)`. 공고 하나에 주택형이 여러 개면 여러 행입니다. 공고 단위 컬럼(단지명·일정 등)은 주택형 행마다 반복됩니다.

묶음	컬럼
식별	`source` (applyhome/lh/sh) · `external_id` (청약홈 PBLANC_NO / LH PAN_ID / SH 글 seq) · `model_no` (공고 안 주택형 순번) · `unit_type` (주택형) · `house_type` (APT 등 유형)
공고 정보	`complex_name` · `region_code` / `region_name` (청약홈 지역코드 체계) · `supply_type` · `rent_type` / `rent_type_code` · `house_code` · `house_detail_code` · `search_house_code` · `zip_code` · `address` · `notice_url` · `homepage_url` · `developer` · `constructor` · `contact_phone` · `newspaper` · `notice_status` (LH 공고 상태)
면적 · 세대	`exclusive_area_sqm` (전용) · `unit_size_sqm` (공급, API가 줄 때만) · `contract_area_sqm` · `unit_group` · `unit_count` · `general_supply_count` · `special_supply_count` · `special_supply_detail` (JSON) · `total_household_count`
금액 (원)	`sale_price` (분양가) · `subscription_deposit` (청약 신청금) · `rent_deposit` · `monthly_rent` (LH 공급정보, 대부분 "공고문 참조"라 비어 있음)
일정	`recruit_date` · `receipt_start` / `receipt_end` · `special_receipt_start` / `end` · `announce_date` · `contract_start` / `end` · `move_in_date` · `schedule` (APT 순위·지역별 접수일 JSON)
규제 (APT만)	`is_speculative_area` · `is_adjusted_area` · `is_price_capped` · `is_public_housing_district` · `is_redevelopment` · `is_large_housing_site` · `is_public_housing_special_act` · `is_metro_private_public_district`
원본	`raw` — 청약홈 `{"notice": 공고 상세 API 행, "unit": 주택형 API 행}`, LH `{"notice": 목록 API 행, "unit": 공급정보의 주택형 행}`. SH는 API가 없어 비어 있음
미사용	`eligibility` — 자격요건. API에 없어서 비어 있음

source	행	공고	비고
applyhome	25,860	6,065	raw 100%
lh	100,200	21,327	raw 100%. 공급정보에 주택형 표가 없는 공고는 공고 1행
sh	22	22	접수기간을 공고문에서 찾은 모집공고만, 공고 단위 1행

3-2. historical_results — 경쟁률 · 당첨가점 · 특별공급 (346,328행, 23컬럼)

키: `(external_id, unit_type, result_type, category_label)`. 청약홈 공고만 있습니다 (LH·SH는 결과 API가 없음).

result_type	행	category_label 예	채워지는 컬럼
경쟁률	68,402	해당지역 1순위, 기타경기 2순위, 사후 …	`supply_count` · `apply_count` · `competition_rate` · `is_undersubscribed` · `raw_cmpet_rate` (원문, "(△209)" 등)
당첨가점	29,766	해당지역, 기타지역 …	`avg_score` · `cutoff_score` · `top_score` — 점수가 모두 "-"인 15,175행(발표 전·추첨제)도 포함
특별공급	248,160	신혼부부 - 해당지역, 기관추천, 이전기관 …	`supply_count` · `apply_count` · `competition_rate` · `result_status` · `detail` — 배정·신청이 0인 조합 132,464행도 포함

공통 컬럼: `external_id` · `unit_type` · `model_no` · `house_type` · `reside_secd` · `subscrpt_rank` · `region_code` · `year` (당첨자 발표 연도, 공고 상세와 연결해 채움) · `exclusive_area_sqm` · `raw` (API 행 원본). **주의:** `competition_rate` 는 API 원문이 비어 있거나 미달 표기일 때 신청 ÷ 공급으로 채운 값이 섞여 있습니다 (원문은 `raw_cmpet_rate`).

3-3. lh_notice_details — LH API 응답 원본 (21,327행)

키: `pan_id` (= listings.external_id). `notice` (목록 행) · `detail` (상세정보: 단지·일정·첨부파일 dsAhflInfo) · `supply` (공급정보: 주택형 표 dsList01~04) 세 JSON을 그대로 보관하고, `error` 가 있으면 다음 수집 때 다시 받습니다. **21,327건 모두 완료**했습니다 (2010-10 ~ 현재, 오류 0건). 이후에는 새 공고와 목록 행(상태·일정)이 바뀐 공고만 다시 받습니다.

3-4. loan_products (280행) · loan_rate_options (682행)

테이블	컬럼
`loan_products` 키 (product_type, fin_co_no, fin_prdt_cd)	`product_type` (주택담보대출 111 / 전세자금대출 58 / 개인신용대출 111) · `fin_group` · `company_name` · `name` · `join_way` · `loan_limit_text` · `extra_cost_text` · `prepayment_fee_text` · `delay_rate_text` · `disclosure_month` · `disclosure_start` / `end` · `submitted_at` · `credit_product_type` (`_name`) · `cb_name` · `interest_rate_min` / `max` (옵션 중 최저·최고) · `raw`
`loan_rate_options` `loan_product_id` 로 연결	`collateral_type` / `collateral_code` · `repay_type` / `repay_code` · `rate_type` / `rate_type_code` · `rate_min` · `rate_max` · `rate_avg` · `credit_grade_rates` (신용점수 구간별 금리 JSON) · `raw`

3-5. bankbook_stats — 청약통장 통계 (2,041,305행) · subscription_stats — 청약 신청·당첨자 통계 (2,786행)

테이블	kind	행	기간	컬럼
`bankbook_stats`	전체	5,572	2010-01 ~ 2026-07	`year_month` · `sido_name` · `area_code` / `area_name` (청약지역) · `item_code` / `item_name` (통장 종류) · `balance_code` / `balance_name` (평형) · `period_code` / `period_name` (가입기간 17구간) · `rank1_count` · `rank2_count` · `account_count` · `extra`
	통장별	85,744	2015-01 ~ 2026-07	
	가입기간별	1,949,989	2015-01 ~ 2026-07	
`subscription_stats`	지역별 신청자 / 당첨자	876 × 2	2020-02 ~ 2026-08	`year_month` · `area_code` / `area_name` (17개 시도) · `age_30` · `age_40` · `age_50` · `age_60` · `special_supply_count` · `special_apply_count` · `special_rate` · `general_supply_count` · `general_apply_count` · `general_rate` · `special_rate_text` / `general_rate_text` (원문, 미달이면 "(△178)") · `raw`
	연령별 신청자 / 당첨자	79 × 2		
	지역별 경쟁률	876		

기능마다 주는 필드가 달라 해당 없는 컬럼은 NULL입니다. bankbook_stats는 행이 많아 raw JSON 대신 응답 필드를 전부 컬럼으로 옮기고, 모르는 필드만 `extra` 에 남깁니다 (현재 0행). 지역별 경쟁률의 미달 행은 숫자 컬럼을 비워 두고 원문만 남깁니다.

3-6. sh_notices — SH 게시판 (2,621행: 분양 917 · 임대 1,704)

키: `(board, seq)` . `number` · `title` · `department` · `posted_date` · `views` · `attachment_count` · `attachments` (파일명) · `attachment_links` (파일명 + 원본 다운로드 링크) · `body_text` (본문) · `url` . 청약 공고 외에 당첨자 발표·계약 안내 같은 공지도 함께 있습니다.

4. 공고문·첨부에서 읽은 테이블

계약금은 API에 없고 공고문 PDF 안에만 있어서, **접수 예정·접수 중인 공고**의 공고문과 첨부 파일을 읽어 만듭니다. 금액은 모두 원문에 적힌 숫자이고, 어느 줄에서 읽었는지를 같이 저장합니다.

테이블	키	컬럼과 의미
`notice_documents` 302행	(source, external_id)	`file_name` · `url` · `status` (ok / no_document / error) · `error` · `page_count` · `text` (PDF에서 뽑은 텍스트, 첨부 PDF는 뒤에 이어 붙임). source: applyhome 13 · lh 125 · sh 39 · **lh_list** 125 (주택목록 엑셀 읽기 상태, text 없음)
`lh_rental_houses` 1,692행 (공고 18건)	(pan_id, file_name, sheet, seq)	LH 매입임대·든든전세 주택목록 엑셀의 주택 1호. `address` · `dong` · `ho` · `house_label` · `house_kind` · `exclusive_area_sqm` · `common_area_sqm` · `rooms` · `floor` · `rent_deposit` · `monthly_rent` · `condition_group` · `conditions` (자격별 조건 전체 JSON)
`notice_costs` 967행	(source, external_id, model_no) = listings 행과 1:1	`contract_deposit` (계약금, 금액이 없는 공고는 NULL) · `contract_ratio` · `deposit_total` (표의 계) · `balance` · `method` (어떻게 구했는지) · `doc_label` (공고문 표기) · `source_line` (**근거 원문 줄**)
`notice_option_costs` 219행 (분양 공고 6건)	(source, external_id, seq)	발코니 확장·선택품목 가격표의 줄. `kind` · `label` · `total` · `contract` · `balance` · `source_line`. 주택 계약금과 합치지 않음

notice_costs.method — 계약금을 구한 방법

method	의미	근거 수준
공고문 표	표의 주택형 표기가 완전히 같은 줄 (계 = 계약금 + 잔금)	확실
공고문 표(면적 일치)	표 줄 안에 같은 전용면적 숫자가 있는 줄	사실상 확실
공고문 표(주택형 숫자 일치)	주택형 앞 숫자와 타입 문자가 같은 줄	대부분 확실
공고문 표(면적 추정 연결)	API 주택형이 깨져 전용면적 정수로 연결	연결만 추론
공고문 표(가격 일치)	분양 행: 표의 계가 분양가와 가장 가까운 줄	사실상 확실
공고문 정액 / 공고문 비율	"계약금 1,000만원" / "계약금(10%)" × 분양가	원문 문장
공고문 표·최솟값	주택형을 연결하지 못해 그 공고의 가장 낮은 계약금	"최소 금액"
주택목록 보증금 / 주택형 최솟값 / 최솟값	LH 주택목록의 임대보증금 전액 (같은 주택 / 같은 주택형 / 공고 최저)	보증금 전액 기준
공고문 보증금 최솟값 / 본인부담 보증금	보증금만 있는 표의 최솟값 / 전세임대 본인부담 정액	원문
배정 시 안내 / 주택 선택 후 결정	공고에 금액이 없음 (예비입주자 모집 / 전세임대 비율)	금액 NULL

공고문에서 찾지 못하고 분양가가 있으면 가공 레이어가 "추정(분양가 10%)"로 채웁니다 (notice_costs에는 없음). 규칙은 `document_client.py`, 테스트는 `tests/test_document_client.py` · `test_house_list_client.py` (24개 통과).

5. 가공 레이어

원본을 읽어 서비스에 필요한 값을 계산한 뷰입니다. **읽기 전용**이고, 결과를 바꾸려면 `app/db/views.py`의 SQL을 고친 뒤 `python -m app.db.views`로 다시 만듭니다. 원본은 영향받지 않습니다.

뷰	종류	내용
`listing_features` 126,082행	구체화 뷰 수집 후 재계산	listings 1행 = 1행. 추가 컬럼: `sigungu`(주소에서 추출) · `is_private` · `area_band`(평형 구간) · `unit_size_sqm`(공급면적 — API 값이 없으면 같은 단지 APT의 같은 주택형 값) · `unit_size_source` · `exclusive_pyeong` · `supply_pyeong` · `price_per_exclusive_sqm` · `price_per_exclusive_pyeong` · `price_per_supply_pyeong` · `contract_deposit_10pct`
`recommendable_listings` 841행 (공고 119건, 그중 821행에 계약금 금액)	일반 뷰 조회할 때마다 계산	listing_features 중 **접수 종료일 ≥ 오늘(한국 날짜)**, 취소공고 제외, LH "접수마감" 제외. 추가 컬럼: `receipt_status`(접수 예정/접수 중) · `days_until_receipt` · `contract_deposit`(notice_costs 값, 없고 분양가가 있으면 분양가 × 10%) · `contract_deposit_method` · `contract_ratio`
`cutoff_stats` 2,344행	구체화 뷰	당첨가점 중 **커트라인이 있는 행만**(14,591행) 집계. 지역 × 시군구 × 평형 구간 × 민영 여부 × 거주 구분 × 연도별 `sample_count` · `avg_cutoff` · `median_cutoff` · `min_cutoff` · `max_cutoff` · `avg_winner_score`
`competition_stats` 9,676행	구체화 뷰	경쟁률 행(68,402행) 집계. 유형 × 지역 × 시군구 × 평형 × 구분 × 연도별 `total_supply` · `total_apply` · `overall_rate` · `median_rate` · `undersubscribed_ratio`

서비스에서 쓰는 곳

API	읽는 데이터
`GET /recommendations/{user_id}`	recommendable_listings의 계약금과 `user_profiles.available_assets` 비교 → eligible, cutoff_stats로 당첨 가능 구간 추정
`GET /listings/affordable?assets=`	recommendable_listings 중 계약금 ≤ 보유 자산 (`include_unknown=true`면 금액 없는 행 포함)
`GET /listings/` · `/listings/{id}` · `/loans/`	listings · loan_products 원본

> **아직 가공 레이어가 없는 원본** — 청약통장 통계(bankbook_stats), 청약 신청·당첨자 통계(subscription_stats), 옵션 계약금(notice_option_costs), LH 주택별 월임대료(lh_rental_houses), 추가된 파인 필드는 원본에만 있고 뷰·API에서는 아직 쓰지 않습니다. 가공 레이어를 설계할 때 이 원본들을 재료로 쓰면 됩니다.

6. 테이블 연결 방법

연결	조인 조건
listings ↔ historical_results	`h.external_id = l.external_id AND h.unit_type = l.unit_type` (청약홈 공고만)
listings ↔ notice_costs	`c.source = l.source AND c.external_id = l.external_id AND c.model_no IS NOT DISTINCT FROM l.model_no`
listings(lh) ↔ lh_notice_details	`d.pan_id = l.external_id`
listings(lh) ↔ lh_rental_houses	`h.pan_id = l.external_id` (주택 단위는 주소 + 호로 연결 — notice_costs.source_line에 파일/시트/순번)
listings(sh) ↔ sh_notices	`n.seq = l.external_id`
listings ↔ notice_documents	`d.source = l.source AND d.external_id = l.external_id` (주택목록 상태는 `d.source = 'lh_list'`)
listings ↔ notice_option_costs	`o.source = l.source AND o.external_id = l.external_id` (공고 단위)
loan_products ↔ loan_rate_options	`o.loan_product_id = p.id`
listings ↔ 통계 (지역)	subscription_stats: `s.area_code = l.region_code` (17개 시도, 같은 청약홈 지역코드) bankbook_stats: `area_code` 가 청약지역 4구분(100 서울 / 400 인천·경기 / 700 5대광역시 / 900 기타)이라 지역코드를 묶어서 연결

```
-- 지금 추천 대상 공고와 계약금, 근거
SELECT r.source, r.complex_name, r.unit_type, r.receipt_status, r.contract_deposit, r.contract_deposit_method,
c.source_line
FROM recommendable_listings r
LEFT JOIN notice_costs c ON c.source = r.source AND c.external_id = r.external_id AND c.model_no IS NOT DISTINCT
FROM r.model_no
ORDER BY r.contract_deposit;

-- 공고 하나의 주택형별 경쟁률·커트라인
SELECT l.unit_type, h.result_type, h.category_label, h.competition_rate, h.cutoff_score
FROM listings l JOIN historical_results h ON h.external_id = l.external_id AND h.unit_type = l.unit_type
WHERE l.external_id = '2026000202' ORDER BY 1, 2, 3;

-- 컬럼에 없는 API 필드 꺼내기 (raw)
SELECT external_id, raw -> 'notice' ->> 'PAN_SS' AS lh_상태, raw -> 'unit' FROM listings WHERE source = 'lh'
LIMIT 5;

-- LH 매입임대 주택별 임대조건 (자격별 조건 포함)
SELECT pan_id, address, ho, exclusive_area_sqm, rent_deposit, monthly_rent, condition_group, conditions FROM
lh_rental_houses;

-- 서울의 월별 일반공급 경쟁률과 1순위 통장 수
SELECT s.year_month, s.general_rate, s.general_rate_text, b.rank1_count
FROM subscription_stats s
LEFT JOIN bankbook_stats b ON b.kind = '전체' AND b.year_month = s.year_month AND b.area_code = '100' AND
b.item_code = '04'
WHERE s.kind = '지역별 경쟁률' AND s.area_code = '100' ORDER BY 1 DESC;
```

DBeaver 연결: Host `localhost` · Port `5432` · Database / User / Password `cheongyak`. 원본·추출 테이블은 `Schemas → public → Tables`, 가공 레이어는 `Materialized Views` 와 `Views` 에 있습니다.

7. 갱신 규칙과 알아둘 점

7-1. 구조와 데이터 모두 완료

항목	상태	설명
수집 구조 (출처 5곳 → 테이블)	완성	모든 출처가 매일 오전 6시 자동 수집에 들어 있고, 새 공고·새 달 통계는 자동으로 들어옵니다
LH 전체 기간 (2010-10 ~ 현재)	완료	2026-10-02에 21,327건 모두 받았습니다 (오류 0건). 공공데이터포털 개발 계정은 API별 하루 10,000건 한도가 있어 처음부터 다시 받으면 2~3일 걸립니다 → 팀원은 덤프 파일로 받으세요 ("청약플랫폼_덤프파일_가이드.pdf")
청약 신청·당첨자 통계	완료	2026-10-01 활용신청 승인 후 전체 기간 수집

7-2. 지켜야 할 규칙

원본·추출 테이블을 DBeaver에서 직접 고치지 마세요. 다음 수집 때 API·공고문 값으로 다시 덮어씁니다. 보정이 필요하면 보정용 테이블을 따로 만들고 가공 레이어에서 합치세요.

테이블 구조를 바꿀 때 — 마이그레이션 도구(Alembic)가 아직 없어서, `models/*.py`에 컬럼을 추가한 뒤 `ALTER TABLE … ADD COLUMN`을 직접 실행해야 합니다 (`create_all`은 이미 있는 테이블을 바꾸지 않음). DB 데이터는 Git으로 공유되지 않고, 같은 코드를 실행하면 같은 DB가 만들어집니다.

7-3. 구조상 한계

항목	내용
SH	공식 API가 없어 홈페이지 게시판을 읽습니다. 사이트 구조가 바뀌면 수집 코드를 고쳐야 하고, 평형별 행·임대조건은 없습니다 (공고 단위 1행)
임대보증금·월임대료	LH API가 국민·행복·영구임대 등에서 "공고문 참조"로만 줘서 listings에는 대부분 비어 있습니다. 매입임대는 lh_rental_houses에 있습니다
계약금	접수 예정·접수 중인 공고만 읽습니다 (지난 공고는 대상 아님). HWP만 있는 공고는 읽지 못합니다
자격요건 · 추첨제 가점	어떤 API에도 없습니다 (`listings.eligibility`는 비어 있음)
오피스텔 공급면적	API에 없어 비어 있습니다. 가공 레이어도 추정값을 넣지 않습니다
수집하지 않는 것	LH 토지·상가 공고, 파인 예·적금·연금 상품 (주택·대출과 무관해서 제외)
API 키	`backend/.env`의 `APPLYHOME_API_KEY` (청약홈 4개 서비스 + LH 3개, 같은 공공데이터포털 계정) · `FINE_API_KEY`. 서비스마다 포털에서 따로 활용신청해야 하고, 신청 전에는 401/403이 옵니다

수치는 2026-10-02 `cheongyak` DB 조회 결과입니다. 매일 수집으로 행 수는 조금씩 늘고, recommendable_listings는 접수가 끝난 공고가 빠지면서 줄기도 합니다. 테이블 크기에는 갱신 과정에서 생긴 여유 공간이 포함돼 있습니다.